miniengine

API-referentie

HTTP, websocket en SDK voor Mini Engine 0.2: kamers, rollen, acties en accounts.

Versie 0.2, oktober 2026. Routes onder `/api/v1`.

Inhoud

Zo lees je dit 3

Begrippen 4

- Rollen en wat ze zien 4
- Hostmodi en runtimes 4
- Versies en action-ID's 5

Authenticatie 6

HTTP: games 7

HTTP: kamers 9

- Maken, deelnemen en lezen 9
- Spelverloop 11
- Beheer 13

HTTP: accounts 15

Websocket 17

Ingebouwde serveracties 19

SDK-client 20

Manifest en instellingen 23

Grenzen en rate limits 25

Foutformaat en foutcodes 26

Zo lees je dit

Deze referentie beschrijft de publieke interface van Mini Engine 0.2: de HTTP-routes, het websocketprotocol, de TypeScript-SDK en het gamepackage. Alles is afgeleid van de broncode en de contractdocumenten van versie 0.2.

Basisadres en versies

De engine luistert standaard op `127.0.0.1:3000` (instelbaar met `MINI_ENGINE_BIND`). Alle API-routes staan onder `/api/v1`; dat is de routeversie, los van de engineversie `0.2.0`. Alleen `/api/health` staat daarbuiten. Voorbeelden gebruiken relatieve paden; zet er je eigen basisadres voor.

Verzoeken en antwoorden

- Verzoeken met een body sturen JSON met `Content-Type: application/json`. Onbekende velden worden afgewezen.
- Antwoorden zijn JSON, behalve de HTML-rolpagina's, de gameassets en routes die `204 No Content` geven.
- Elke fout heeft dezelfde vorm: `{"error":{"code":"…","message":"…"}}`. De `code` is stabiel; de `message` is Nederlands en bedoeld voor mensen.
- Kamercodes zijn niet hoofdlettergevoelig. `tafels` en `TAFELS` zijn dezelfde kamer.

Notatie

Notatie	Betekenis
`{code}`	Een padparameter, hier de kamercode van zes letters.
`HOST_SECRET`	Een plaatsvervanger voor een geheim token van 64 tekens. Echte tokens staan nooit in dit document.
`…`	Ingekort voorbeeld. Het echte antwoord bevat meer velden of langere waarden.
Toegang	Welke sleutel de route verwacht: openbaar, een kamertoken van een bepaalde rol of een accounttoken.

Vertrouwen. Lokale gamepackages draaien als vertrouwde code op dezelfde origin als de engine. Ze kunnen kamertokens en de demo-accounttoken lezen. Installeer alleen packages die je vertrouwt. Uploads van onbekende makers bestaan in 0.2 niet.

Begrippen

Eén kamer, één spel, een handvol apparaten. De server is het middelpunt: hij bewaart de stand, controleert wie wat mag en stuurt iedere rol alleen wat die rol mag zien.

Kamers

Een kamer hoort bij precies één game. Hij heeft een code van zes hoofdletters uit `ABCDEFGHJKLMNPQRSTUVWXYZ` (geen I en O, geen cijfers) en een status die de levenscyclus volgt:

```
lobby → running → finished → lobby
```

Nieuwe spelers komen in de lobby binnen, of tijdens `running` als late joining is toegestaan. Terug naar de lobby geeft een nieuwe `round_id`, zet readiness en spelstate terug en begint een nieuwe deduplicatieperiode. Een kamer verloopt na twaalf uur zonder activiteit; verbonden apparaten houden hem levend. Kamers, accounts en resultaten staan in SQLite en overleven een herstart. Aanwezigheid begint na een herstart als offline.

Rollen en wat ze zien

Elk kamertoken hoort bij één rol. De rol bepaalt welke delen van de state in de snapshot staan.

Rol	Wie	Ziet
`host`	De maker van de kamer. Blijft `host`, ook als hij meespeelt.	`public_state`, `host_state`, alle `player_states`; als hij meespeelt ook zijn eigen `private_state`.
`player`	Iemand die met een naam is binnengekomen.	`public_state` en alleen zijn eigen `private_state`.
`display`	Het gedeelde scherm, bijvoorbeeld de tv.	Alleen `public_state`. Nooit private state.

Namen, aanwezigheid, readiness, eventuele account-ID's, instellingen en de kamerconfig zijn voor alle rollen zichtbaar. De openbare info-route toont geen namen en geen spelstate.

Hostmodi en runtimes

De hostmodus ligt vast bij het maken van de kamer. De game bepaalt welke modi mogen.

Hostmodus	De maker	Spelverloop
`moderator`	Host zonder spelersstoel; `player_id` is `null`.	Een aparte spelleider bedient het spel en heeft zelf geen readiness.
`playing`	Host met spelersstoel.	De maker speelt mee en houdt zijn beheerrechten.
`hostless`	Host met spelersstoel.	Deelnemers bedienen het servergame; het tabblad van de maker hoeft niet verbonden te blijven.

De **runtime** zegt waar de spelregels draaien.

- `runtime:"server"`: de regels draaien in Rust op de server. In 0.2 geldt dat alleen voor de ingebouwde games Common Dice (alle drie de modi, standaard `hostless`) en Jeopardy (alleen `moderator`). Een manifestveld alleen installeert geen nieuwe serverruntime.
- `runtime:"host"`: een vertrouwde browserhost verwerkt acties en publiceert de volledige state. Alleen `moderator` is toegestaan en het hosttabblad moet verbonden blijven.

Versies en action-ID's

Elke kamer heeft een `version`. Gameplay, instellingen, config en lifecycle kunnen die verhogen. Aanwezigheid, binnenkomen, vertrekken en readiness geven ook een snapshot, maar verhogen de versie niet. Filter snapshots dus niet weg alleen omdat de versie gelijk bleef.

Wijzigingen die een gelijktijdige schrijver kunnen overschrijven, sturen `expected_version` mee: state publiceren, instellingen vervangen en de config aanpassen. Klopt de versie niet meer, dan volgt `409 version_conflict`. Lees de nieuwste snapshot en beoordeel de handeling opnieuw.

Acties op een servergame mogen een `action_id` meekrijgen: 1–64 tekens zonder controletekens. De server onthoudt per kamer de laatste 256 verwerkte ID's, gekoppeld aan actor en ronde. Een herhaling binnen dat venster slaagt zonder tweede effect en zonder versieverhoging. De SDK maakt standaard een unieke ID per actie en herhaalt opdrachten nooit vanzelf.

Authenticatie

Er zijn twee soorten sleutels die los van elkaar staan: kamertokens voor één kamer en accounttokens voor een gedeeld account. Beide gaan als bearer in de header, nooit in de URL.

```
Authorization: Bearer <token>
```

Kamertokens

Maken en deelnemen geven een token van 64 tekens terug, met de bijbehorende rol. De server bewaart alleen een SHA-256-hash. Er is één token voor de host, één per speler en één voor het gedeelde scherm.

- Eén token heeft één actieve websocketverbinding. Een nieuwe verbinding met hetzelfde token vervangt de oude; die krijgt `session_replaced`.
- Vertrekken of verwijderd worden trekt het spelerstoken in. Een kamer verwijderen trekt alles in.
- De SDK bewaart kamertokens in `sessionStorage` onder `mini-engine:CODE:ROLE`.

Displaytokens

Het token voor het gedeelde scherm zit in het antwoord van `POST /api/v1/rooms` als `display_token`, tenzij het scherm is uitgeschakeld. Het scherm opent `/room/CODE/display#TOKEN`: de schil bewaart het fragment en haalt het weg voor het gameframe laadt, dat alleen `?room=CODE` krijgt.

De host kan een nieuw displaytoken opvragen; het vorige werkt dan niet meer. Het scherm uitschakelen trekt het token blijvend in. Na opnieuw inschakelen vraagt de host een nieuw token op; het oude komt niet terug.

Accounttokens

Accounts zijn optioneel; gastspel werkt zonder. Registreren en inloggen geven een willekeurig sessietoken van 64 tekens dat dertig dagen geldig is en een herstart overleeft. Elke login maakt een eigen sessie; uitloggen trekt alleen de gebruikte sessie in.

Stuur een accounttoken mee bij het maken of binnenkomen van een kamer om de spelersstoel aan het account te koppelen. De server controleert het token; een ongeldig token geeft `401`. De body kan geen `account_id` opgeven. Eén account bezet hoogstens één actieve stoel per kamer.

Origin

Verzoeken met een `Origin`-header worden alleen geaccepteerd als die origin gelijk is aan de host van het verzoek of in `MINI_ENGINE_ORIGINS` staat (kommagescheiden). Anders volgt `403 origin_forbidden`. Elk antwoord krijgt `no-store`, `nosniff`, `no-referrer` en een strikte content security policy die alleen `'self'` toestaat.

HTTP: games

Openbare routes die de geïnstalleerde games en hun bestanden beschrijven. Geen token nodig.

`GET /api/health`

Toegang openbaar

Status en de gebouwde engineversie.

Antwoord 200

```
{"status": "ok", "version": "0.2.0"}
```

`GET /api/v1/games`

Toegang openbaar

Alle geïnstalleerde games met metadata en instellingenschema. De velden staan bij *Manifest en instellingen*.

Antwoord 200

```
{
  "games": [
    {
      "id": "common-dice",
      "title": "Common Dice",
      "description": "Eén worp voor de hele tafel. …",
      "version": "2.0.0",
      "min_players": 1,
      "max_players": 100,
      "has_display": true,
      "supports_late_join": true,
      "default_settings": {
        "dice_count": 5, "sides": 6, "roll_permission": "everyone", "theme": "ivory"
      },
      "settings_schema": {
        "type": "object", "properties": {…}, "required": […], "additionalProperties": false
      },
      "runtime": "server",
      "host_modes": ["moderator", "playing", "hostless"],
      "default_host_mode": "hostless"
    }
  ]
}
```

`GET /api/v1/games/{id}`

Toegang openbaar

De metadata van één game, in dezelfde vorm als een item uit de lijst.

Fouten: `404 game_not_found`

`GET /api/v1/games/{id}/{role}`

Toegang openbaar

De HTML-pagina van de game voor `host`, `player` of `display`, als `text/html`. De launcher laadt deze pagina in het gameframe.

Fouten: `404 game_not_found`, `404 game_page_not_found`

`GET /games/{id}/assets/{path}`

Toegang openbaar

Een publiek bestand uit de map `assets/` van de game, met een MIME-type op basis van de extensie. Paden met lege delen, `.`, `..` of backslashes worden geweigerd. `data.json` is nooit een publiek asset.

Fouten: `404 asset_not_found`

Pagina's en SDK buiten de API

Pad	Inhoud
`/`	Tijdelijke doorverwijzing naar `/demo`, de startpagina.
`/demo`, `/demo/join`, `/demo/account`, `/demo/new/{id}`	Werkbankpagina's om games te kiezen, kamers te maken en binnen te komen.
`/room/{code}/{role}`	De kamerschil voor `host`, `player` of `display`, met het gameframe erin.
`/sdk/mini-engine.js`	De door de engine geserveerde SDK-build als ES-module.
`/sdk/mini-engine.d.ts`	De bijbehorende TypeScript-declaraties, als platte tekst.

HTTP: kamers

Routes om kamers te maken, binnen te komen en het spel te bedienen. De meeste routes geven de bijgewerkte snapshot van jouw rol terug, zodat je meteen kunt tekenen.

Maken, deelnemen en lezen

`POST /api/v1/rooms`

Toegang gast, of een accounttoken om de stoel van de maker te koppelen

Maakt een kamer en geeft de hostcredentials terug. Alleen `game_id` is verplicht.

- `max_players`: standaard het maximum van de game, afgetopt op 24 en niet lager dan het minimum.
- `settings`: wordt samengevoegd met de standaardinstellingen van de game en daarna tegen het schema gecontroleerd.
- `config`: weggelaten velden worden de standaardmodus van de game, `display_enabled` volgens `has_display` en `allow_late_join:false`.
- `name`: naam van een meespelende maker; standaard `Spelleider`.

Verzoek

```
{
  "game_id": "common-dice",
  "name": "Dani",
  "max_players": 12,
  "settings": {"dice_count": 5, "sides": 6},
  "config": {"host_mode": "hostless", "display_enabled": true, "allow_late_join": false}
}
```

Antwoord 201

```
{
  "code": "TAFELS",
  "token": "HOST_SECRET",
  "role": "host",
  "player_id": "8c0e6c2a-…",
  "display_token": "DISPLAY_SECRET"
}
```

Een moderator krijgt `"player_id": null`. Bij een uitgeschakeld scherm ontbreekt `display_token`.

Fouten: `400 invalid_request` (spelersaantal, instellingen, modus of late joining die de game niet ondersteunt; browsergames vereisen `moderator`), `400 invalid_json`, `401 unauthorized` (ongeldig accounttoken), `404 game_not_found`, `429 rate_limit`, `503 room_limit`

`POST /api/v1/rooms/{code}/join`

Toegang gast, of een accounttoken om de stoel te koppelen

Komt de kamer binnen als speler. Namen zijn 1–24 tekens zonder controletekens en uniek binnen de kamer, ongeacht hoofdletters.

Verzoek

```
{"name": "Sander"}
```

Antwoord 201

```
{"code": "TAFELS", "token": "PLAYER_SECRET", "role": "player", "player_id": "3f2c9d71-…"}
```

Fouten: `400 invalid_request` (naam), `401 unauthorized`, `404 room_not_found`, `409 room_started`, `409 room_full`, `409 name_taken`, `409 already_joined` (dit account zit al in de kamer; hervat de bestaande sessie), `429 rate_limit`

`GET /api/v1/rooms/{code}/info`

Toegang openbaar

Wat je moet weten voordat je binnenkomt. Geen namen, geen spelstate.

Antwoord 200

```
{
  "code": "TAFELS",
  "game_id": "common-dice",
  "status": "lobby",
  "player_count": 3,
  "max_players": 12,
  "config": {"host_mode": "hostless", "display_enabled": true, "allow_late_join": false},
  "runtime": "server"
}
```

Fouten: `404 room_not_found`

`GET /api/v1/rooms/{code}`

Toegang elk kamertoken

De snapshot voor jouw rol. Dezelfde vorm komt over de websocket als `{"type":"snapshot","room":…}`. Hieronder de snapshot van een speler in Common Dice.

Antwoord 200


```
{
  "code": "TAFELS",
  "game_id": "common-dice",
  "status": "running",
  "version": 7,
  "max_players": 12,
  "settings": {
    "dice_count": 5, "sides": 6, "roll_permission": "everyone", "theme": "ivory"
  },
  "host_connected": true,
  "players": [
    {"id": "8c0e6c2a-…", "name": "Dani", "connected": true, "ready": true,
     "account_id": null},
    {"id": "3f2c9d71-…", "name": "Sander", "connected": true, "ready": true,
     "account_id": "b71e0c55-…"}
  ],
  "public_state": {
    "results": [5, 3, 6, 2, 6], "roller": "Sander", "roller_id": "3f2c9d71-…",
    "sequence": 4, "configuration": "5d6", "held": [], "rolled_at": 1792656000000,
    "history": […]
  },
  "you": {"role": "player", "player_id": "3f2c9d71-…"},
  "runtime": "server",
  "config": {"host_mode": "hostless", "display_enabled": true, "allow_late_join": false},
  "round_id": "9b1d4f0e-…",
  "private_state": null
}
```


De host krijgt daarnaast `host_state` en `player_states`; een display krijgt geen `private_state`. `host_connected` is alleen aanwezigheid: serveracties hangen er niet van af.

Fouten: `401 unauthorized`, `404 room_not_found`

`GET /api/v1/rooms/{code}/game-data`

Toegang hosttoken

De private `data.json` van de game, bijvoorbeeld vragen en antwoorden. Alleen de host kan dit lezen.

Fouten: `401 unauthorized`, `403 forbidden`, `404 room_not_found`

Spelverloop

`POST /api/v1/rooms/{code}/ready`

Toegang deelnemer met een stoel, alleen in de lobby

Zet jouw readiness. Starten kan pas als iedere deelnemer klaarstaat. Verhoogt de versie niet.

Verzoek


```
{"ready": true}
```

Antwoord 200

De bijgewerkte snapshot.

Fouten: `403 forbidden` (geen spelersstoel, zoals een moderator of display), `409 room_started`

`POST /api/v1/rooms/{code}/actions`

Toegang speler; bij een servergame ook de host als de regels dat toestaan

Stuurt een spelactie. De naam is 1–64 letters, cijfers, `_`, `.` of `-`; `data` is hoogstens 8 KiB en standaard `null`. Bij een servergame verwerkt Rust de actie en slaat het resultaat op voordat het zichtbaar wordt. Bij een hostruntime moet de host verbonden zijn; de actie gaat dan naar zijn wachtrij.

Verzoek

```
{"name": "roll", "data": {"held": [0, 2]}, "action_id": "c0a8e1f4-…"}
```

Antwoord 200

De bijgewerkte snapshot.

Fouten: `400 invalid_request` (naam, grootte of `action_id`), `403 forbidden` (display, of een rol die deze actie niet mag), `409 game_not_running`, `409 host_offline`, `409 invalid_game_action`, `409 buzzer_closed`

`PUT /api/v1/rooms/{code}/state`

Toegang hosttoken, alleen bij `runtime:"host"`

Vervangt de volledige state. Weggelaten `host_state` wordt `null`, weggelaten `player_states` wordt `{}`. Sleutels in `player_states` moeten bestaande speler-ID's zijn. De state wordt opgeslagen voordat de nieuwe snapshot uitgaat. Servergames weigeren deze route, ook voor de host.

Verzoek

```
{
  "expected_version": 4,
  "public_state": {"round": 1},
  "host_state": {"answer": "Venus"},
  "player_states": {"3f2c9d71-…": {"cards": [1, 2]}}
}
```

Fouten: `400 invalid_request` (onbekende speler), `403 forbidden`, `409 version_conflict`, `413 state_too_large` (meer dan 64 KiB)

`PUT /api/v1/rooms/{code}/settings`

Toegang hosttoken

Vervangt het hele instellingenobject. De server controleert het schema van de game en daarna de spelregels; bij Common Dice wist een ander aantal of andere zijden de worpen, een ander thema niet.

Verzoek

```
{
  "expected_version": 7,
  "settings": {
    "dice_count": 3, "sides": 20, "roll_permission": "everyone", "theme": "obsidian"
  }
}
```

Fouten: `400 invalid_request`, `403 forbidden`, `409 version_conflict`, `409 teams_locked` (Jeopardy)

`POST /api/v1/rooms/{code}/lifecycle`

Toegang hosttoken

Zet de status door: `lobby → running → finished → lobby`. Naar `running` vereist dat alle deelnemers klaarstaan; Jeopardy vraagt daarnaast een teamkeuze van iedere speler. Afronden van een servergame slaat de accountresultaten op in dezelfde transactie als de kamer.

Verzoek

```
{"status": "running"}
```

Fouten: `403 forbidden`, `409 invalid_transition`, `409 players_not_ready`, `409 teams_required`

Beheer

`PUT /api/v1/rooms/{code}/config`

Toegang hosttoken, alleen in de lobby

Past `display_enabled` en `allow_late_join` aan. De hostmodus ligt vast; onbekende velden worden afgewezen. Late joining kan alleen aan als de game `supports_late_join:true` heeft.

Verzoek

```
{"expected_version": 0, "config": {"display_enabled": false, "allow_late_join": true}}
```

Fouten: `400 invalid_request`, `403 forbidden`, `409 room_started`, `409 version_conflict`

`POST /api/v1/rooms/{code}/display-token`

Toegang hosttoken, scherm ingeschakeld

Maakt een nieuw displaytoken. Het vorige werkt meteen niet meer.

Antwoord 200

```
{"token": "DISPLAY_SECRET"}
```

Fouten: `403 forbidden` (geen host, of scherm uitgeschakeld)

`POST /api/v1/rooms/{code}/leave`

Toegang spelerstoken

Verlaat de kamer en trekt het eigen token in. Antwoord `204` zonder body. De host kan deze route niet gebruiken.

Fouten: `401 unauthorized`, `403 forbidden`

`DELETE /api/v1/rooms/{code}/players/{player_id}`

Toegang hosttoken

Verwijdert een speler, zijn private state en zijn token. Antwoord `204`. De maker kan zijn eigen stoel niet verwijderen.

Fouten: `400 invalid_request` (eigen stoel), `403 forbidden`, `404 player_not_found`

`DELETE /api/v1/rooms/{code}`

Toegang hosttoken

Verwijdert de kamer en sluit alle verbindingen met `room_deleted`. Antwoord `204`.

Fouten: `401 unauthorized`, `403 forbidden`, `404 room_not_found`

HTTP: accounts

Een lokaal account dat alle games op deze engine delen. Geen e-mail, geen externe inlog, geen wachtwoordherstel. Gastspel blijft altijd mogelijk.

Velden en regels

Veld	Regel
`username`	Wordt getrimd en naar kleine letters gezet; daarna 3–24 tekens uit a–z, 0–9 en `_`. Uniek.
`display_name`	Getrimd, 1–24 Unicode-tekens, geen controletekens.
`password`	Precies zoals ingevoerd, 10–128 UTF-8-bytes. Opgeslagen als Argon2id-hash.

`POST /api/v1/accounts/register`

Toegang openbaar

Verzoek

```
{"username": "sander", "password": "dobbelsteen-42", "display_name": "Sander"}
```

Antwoord 201

```
{
  "account": {
    "id": "b71e0c55-…", "username": "sander", "display_name": "Sander",
    "created_at": 1792656000
  },
  "token": "ACCOUNT_SECRET"
}
```

`created_at` is in Unix-seconden. Het token wordt niet automatisch bewaard.

Fouten: `400 invalid_request`, `409 username_taken`, `429 rate_limit`

`POST /api/v1/accounts/login`

Toegang openbaar

Geeft dezelfde vorm als registreren, met status 200. Een onbekende naam en een fout wachtwoord geven dezelfde fout.

Verzoek

```
{"username": "sander", "password": "dobbelsteen-42"}
```

Fouten: `401 unauthorized`, `429 rate_limit`

`GET /api/v1/accounts/me`

Toegang accounttoken

Het accountobject: `{id, username, display_name, created_at}`.

`POST` `/api/v1/accounts/logout`

Toegang accounttoken

Trekt deze sessie in. Antwoord `204`. Kamertokens die al zijn uitgegeven blijven geldig.

`GET` `/api/v1/accounts/stats`

Toegang accounttoken

Statistieken uit afgeronde serverrondes, eenmaal per account en ronde. Er is geen route om zelf scores te posten; eigen browsergames tellen niet mee.

Antwoord 200

```
{
  "totals": {"games_played": 12, "wins": 3},
  "games": [
    {"game_id": "jeopardy", "games_played": 5, "wins": 3,
     "total_score": 4200, "best_score": 1400, "counters": {"correct": 17, "wrong": 6}},
    {"game_id": "common-dice", "games_played": 7, "wins": 0,
     "total_score": 0, "best_score": 0,
     "counters": {"rolls": 41, "dice_rolled": 180, "total": 652}}
  ],
  "recent": [
    {"round_id": "9b1d4f0e-…", "game_id": "jeopardy", "room_code": "TAFELS",
     "player_id": "3f2c9d71-…", "account_id": "b71e0c55-…", "display_name": "Sander",
     "score": 1400, "won": true, "counters": {"correct": 5, "wrong": 1},
     "finished_at": 1792659600}
  ]
}
```

`recent` bevat de laatste twintig resultaten. Tellers tellen alleen gehele getallen op; scores kunnen negatief zijn. Common Dice kent geen winnaars toe.

Alle accountroutes met een token geven `401 unauthorized` bij een ontbrekend, ongeldig of verlopen token.

Websocket

Eén verbinding per kamertoken. De server stuurt na elke wijziging een verse snapshot voor jouw rol; jij stuurt opdrachten en krijgt per opdracht een bevestiging of een fout.

Verbinden

```
GET /api/v1/rooms/{code}/ws   (upgrade naar websocket)
```

Stuur binnen vijf seconden als eerste tekstframe het token. Daarna volgt de eerste snapshot.

```
→ {"type": "auth", "token": "PLAYER_SECRET"}
← {"type": "snapshot", "room": {…}}
```

Alleen tekstframes met JSON; binaire frames geven een fout. Na vijf seconden zonder auth volgt `auth_timeout` en sluit de server. Bestaat de kamer of de game niet, dan faalt de upgrade met een gewone HTTP-fout.

Van client naar server

Elk bericht heeft een `type` en mag een `request_id` van hoogstens 64 tekens meesturen. De server zet die terug in de `ack` of `error`.

type	Velden	Gelijk aan
`action`	`name`, `data`, optioneel `action_id`	`POST …/actions`
`ready`	`ready`	`POST …/ready`
`configure`	`expected_version`, gedeeltelijke `config`	`PUT …/config`
`set_state`	`expected_version`, `public_state`, optioneel `host_state`, `player_states`	`PUT …/state`
`settings`	`expected_version`, volledig `settings`	`PUT …/settings`
`lifecycle`	`status`	`POST …/lifecycle`
`ping`	geen	alleen een `ack`

Van server naar client

type	Wanneer	Vorm
`snapshot`	Na auth, na elke wijziging in de kamer en na een eigen mutatie.	`{"type":"snapshot","room":{…}}`
`ack`	Een opdracht is gelukt.	`{"type":"ack","request_id":"1"}`
`error`	Een opdracht is mislukt, of er ging iets mis buiten een opdracht.	`{"type":"error","request_id":"1","error":{"code":"…","message":"…"}}`

`action`	Alleen voor de host van een hostruntime: een speler stuurde een actie.	`{"type":"action","player_id":"…","name":"vote","data":{…},"request_id":"3"}`
`closed`	De server sluit de verbinding.	`{"type":"closed","reason":"room_deleted"}`

Na een mutatie komt eerst de bijgewerkte snapshot en dan de `ack`; de ack bevat zelf geen state. Bij een hostruntime bevestigt de ack van een speleractie alleen dat de actie in de wachtrij van de host staat, niet dat het spel haar heeft verwerkt.

Een ronde in berichten

```
→ {"type": "ready", "ready": true, "request_id": "1"}
← {"type": "snapshot", "room": {"status": "lobby", "version": 3, …}}
← {"type": "ack", "request_id": "1"}
→ {"type": "action", "name": "roll", "data": {"held": []}, "action_id": "c0a8e1f4-…",
"request_id": "2"}
← {"type": "snapshot", "room": {"status": "running", "version": 5, …}}
← {"type": "ack", "request_id": "2"}
```

Sluitredenen

reason	Betekenis	Wat de SDK doet
`session_replaced`	Hetzelfde token verbond opnieuw, bijvoorbeeld in een ander tabblad.	Stopt; credentials blijven bewaard voor expliciet hervatten.
`session_revoked`	Het token is ingetrokken: vertrokken, verwijderd of het scherm is uitgezet.	Stopt.
`room_deleted`	De host heeft de kamer verwijderd.	Stopt.
`room_expired`	Twaalf uur geen activiteit.	Stopt.
`host_lagged`	De host liep te ver achter op de gebeurtenissen.	Verbindt opnieuw met backoff.
`heartbeat_timeout`	45 seconden geen verkeer van de client.	Verbindt opnieuw met backoff.
`rate_limit`	Te veel berichten per seconde.	Verbindt opnieuw na een extra pauze.
`server_shutdown`	De engine stopt.	Verbindt opnieuw met backoff.

De server stuurt elke vijftien seconden een websocket-ping. Spelers en schermen die achterlopen krijgen gewoon een verse snapshot. Onzekere opdrachten worden nooit automatisch herhaald.

Ingebouwde serveracties

Common Dice en Jeopardy draaien in Rust. Hun pagina's sturen alleen acties en tekenen snapshots. De server kiest de uitkomst; animaties laten die alleen zien.

Common Dice

Instellingen: `dice_count` 1–20, `sides` 4, 6, 8, 10, 12 of 20, `roll_permission` `everyone` of `owner`, `theme` `ivory`, `obsidian` of `candy`. Standaard `{dice_count:5, sides:6, roll_permission:"everyone", theme:"ivory"}`.

Actie	Wie	Wat er gebeurt
`roll` `{held?: [0, 2]}`	Iedere deelnemer; bij `owner` alleen de host.	Alleen tijdens `running`. Gooit de vrije stenen opnieuw. `held` verwijst naar de huidige worp; minstens één steen blijft vrij.
`clear_history` `{}`	Host	Wist de zichtbare worpgeschiedenis.
`start` `{}`, `finish` `{}`	In `hostless` iedere deelnemer, anders de host.	Start of rondt de ronde af.

`roll_permission:"owner"` gaat niet samen met `hostless`. De publieke state bevat `results`, `roller`, `roller_id`, `sequence`, `configuration` (bijvoorbeeld `"5d6"`), `held`, `rolled_at` in epochmilliseconden en hoogstens twintig items in `history`.

Jeopardy

Instelling: `team_count` 2–6, standaard 3. Alleen `moderator`; geen late joining. `team_count` kan alleen in de lobby veranderen, zolang nog niemand een team koos.

Actie	Wie	Wat er gebeurt
`choose_team` `{team_id}`	Speler, in de lobby	Kiest een team. Starten vereist dat iedere speler er een heeft.
`buzz` `{}`	Speler	De eerste geldige buzzer bij een open vraag wint. Anders `409 buzzer_closed`.
`select_clue` `{clue_id}`	Host	Opent een vraag. Het antwoord staat in `host_state.active_answer`.
`judge` `{correct}`	Host	Kent de score één keer toe; bij fout gaat de vraag open voor de andere teams.
`reveal`, `reopen`, `close_clue`, `finish`	Host	Onthult het antwoord, opent opnieuw, sluit de vraag of rondt het spel af.

Lege teams kunnen niet winnen. Ongeldige zetten geven `409 invalid_game_action`.

SDK-client

`@casb02/mini-engine-sdk` versie 0.2.0: een getypeerde client voor browsers en Node.js 22 of nieuwer. Het pakket staat in een private repository en is niet op npm gepubliceerd.

Laden

In een gamepagina importeer je de build die de engine zelf serveert. Buiten de engine installeer je de tarball uit `npm pack`.

```
import { ApiClient, MiniEngine, loadSession } from '/sdk/mini-engine.js';

const code = new URL(location.href).searchParams.get('room') || '';
const credentials = loadSession(code, 'player');
if (!credentials) throw new Error('Open deze pagina via de kamer.');

const client = new MiniEngine(credentials).connect();
client.on('snapshot', room => render(room));
client.on('error', error => showError(error.message));
await client.waitForSnapshot();
await client.ready(true);
// Zodra het spel loopt:
await client.action('roll', { held: [] });
```

ApiClient

`new ApiClient({ baseUrl?, accountToken?, fetch? })`. Zonder `baseUrl` gebruikt hij de huidige origin. Methoden geven een Promise en gooien een `MiniEngineError` bij een HTTP-fout.

Methode	Route	Geeft
`getGames()`	`GET /games`	`{ games: Manifest[] }`
`getGame(id)`	`GET /games/{id}`	`Manifest`
`createRoom(options)`	`POST /rooms`	`RoomCredentials`
`joinRoom(code, name)`	`POST /rooms/{code}/join`	`RoomCredentials`
`roomInfo(code)`	`GET /rooms/{code}/info`	openbare kamerinfo
`getRoom(code, token)`	`GET /rooms/{code}`	`Snapshot`
`gameData(code, token)`	`GET /rooms/{code}/game-data`	de private speldata
`displayToken(code, token)`	`POST /rooms/{code}/display-token`	`{ token }`
`register(input)`	`POST /accounts/register`	`AccountSession`
`login(input)`	`POST /accounts/login`	`AccountSession`
`me()`	`GET /accounts/me`	`Account`
`stats()`	`GET /accounts/stats`	`AccountStats`
`logout()`	`POST /accounts/logout`	`null`; wist `accountToken`

`createRoom` en `joinRoom` sturen het `accountToken` mee als dat is ingesteld. `register` en `login` bewaren het token niet; zet het zelf in `accountToken`. Losse helpers voor de huidige origin: `getGames()`, `createRoom(options)`, `joinRoom(code, name)` en `http(path, options)`, die JSON ophaalt met een optioneel `token`.

MiniEngine

`new MiniEngine({ code, token, baseUrl?, reconnectDelay?, requestTimeout?, WebSocket? })`. Een `RoomCredentials`-object past direct. Standaard: `reconnectDelay` 500 ms, verdubbelend tot hoogstens 15 seconden met wat spreiding; `requestTimeout` 10 seconden.

Lid	Wat het doet
`connect()`	Opent de verbinding en authenticeert. Geeft de client terug.
`resume()`	Verbindt opnieuw na een stop, bijvoorbeeld na `session_replaced`. De laatste snapshot blijft staan tot er een nieuwe is.
`close()`	Stopt, wijst open opdrachten af en verbindt niet opnieuw.
`on(type, fn)`, `once(type, fn)`	Luistert naar een event. Geeft een functie terug die het luisteren stopt.
`waitForSnapshot(predicate?, { timeout?, signal? })`	Wacht op een snapshot die aan de voorwaarde voldoet.
`action(name, data?, { actionId? })`	Stuurt een actie, standaard met een nieuwe unieke `action_id`.
`ready(ready = true)`	Zet readiness.
`configure(config)`	Past `display_enabled` of `allow_late_join` aan met de huidige versie.
`setSettings(settings)`	Vervangt alle instellingen met de huidige versie.
`setState(update)`	Publiceert de volledige state; alleen voor een hostruntime.
`lifecycle(status)`	Zet de status door.
`request(type, payload)`	Stuurt een willekeurig bericht met een eigen `request_id` en wacht op de `ack`.
`room`, `connected`, `code`, `token`	De laatste snapshot, de verbindingsstatus en de credentials.

Alle opdrachten geven een Promise die oplost met de `ack`. Ze worden afgewezen bij een `error`, bij een time-out (`request_timeout`), bij verbreken (`disconnected`) of als er nog geen verbinding is (`not_connected`).

Events

Event	Waarde
`snapshot`	De nieuwe `Snapshot`. De eerste snapshot markeert de client als verbonden.

`action`	Een `ActionEnvelope`; alleen voor de host van een hostruntime.
`status`	`connecting`, `connected`, `reconnecting` of `disconnected`.
`error`	Een `MiniEngineError` met `code` en `message`.
`closed`	De sluitreden als de server definitief sloot.

Sessies

Functie	Wat het doet
`sessionKey(code, role)`	Geeft `mini-engine:CODE:ROLE`.
`saveSession(credentials, storage?)`	Bewaart credentials, standaard in `sessionStorage`.
`loadSession(code, role, storage?)`	Leest ze terug, of `null` als ze ontbreken of niet kloppen.
`removeSession(code, role, storage?)`	Verwijdert ze.

Fouten en typen

`MiniEngineError` breidt `Error` uit met `code` en, bij HTTP, `status`. Bij serverfouten is `code` de foutcode van de engine; daarnaast bestaan clientcodes als `not_connected`, `request_timeout`, `snapshot_timeout`, `disconnected`, `closed` en `missing_base_url`.

De belangrijkste typen: `Json`, `Role`, `HostMode`, `Lifecycle`, `ConnectionStatus`, `RoomConfig`, `Player`, `Snapshot<PublicState, Settings, PrivateState>`, `Manifest`, `SettingsProperty`, `RoomCredentials`, `CreateRoomOptions`, `Account`, `AccountSession`, `AccountStats`, `Ack`, `ActionEnvelope`, `StateUpdate` en `ActionPayloads` voor de ingebouwde acties. `MiniEngine<P, S, V>` geeft de typen van publieke state, instellingen en private state door aan `room` en de events.

Een eigen browserhost

Bij `runtime:"host"` verwerkt de hostpagina de acties en publiceert de hele volgende state. Valideer speler, payload, status en spelregels; vertrouw nooit een identiteit of score uit de clientdata. Zet alle mutaties in één wachtrij en wacht op snapshot en ack voordat de volgende een versie gebruikt.

```
client.on('action', async ({ player_id, name, data }) => {
  if (name !== 'vote' || client.room?.status !== 'running') return;
  const next = tally(client.room.public_state, player_id, data);
  await client.setState({
    expected_version: client.room.version,
    public_state: next,
  });
});
```

De scaffolder maakt een werkend voorbeeld: `create-mini-game <slug> [--directory pad]`.

Manifest en instellingen

Een game is een map in `games/` met een manifest, drie rolpagina's en publieke assets. De mapnaam is de game-ID.

```
games/my-game/
  manifest.json
  host.html
  player.html
  display.html     verplicht bij has_display: true
  assets/          publiek, onder /games/my-game/assets/…
  data.json        optioneel en privé; alleen de host leest dit
```

Manifestvelden

Veld	Type	Betekenis
`id`	string	Gelijk aan de mapnaam.
`title`, `description`, `version`	string	Naam, korte omschrijving en eigen versie van de game.
`min_players`, `max_players`	getal	Grenzen voor `max_players` van een kamer.
`has_display`	boolean	Of er een gedeeld scherm is; dan is `display.html` verplicht.
`default_settings`	object	Standaardinstellingen; moeten aan het schema voldoen.
`settings_schema`	object	Het instellingenschema, zie hieronder.
`runtime`	`host` | `server`	Standaard `host`.
`host_modes`	lijst	Toegestane hostmodi. Standaard `["moderator"]`.
`default_host_mode`	string	Standaard `moderator`.
`supports_late_join`	boolean	Standaard `true` voor oudere manifests. Kies het expliciet voor nieuwe games.

```
{
  "id": "my-game",
  "title": "My Game",
  "description": "Een spel voor vrienden.",
  "version": "1.0.0",
  "runtime": "host",
  "host_modes": ["moderator"],
  "default_host_mode": "moderator",
  "supports_late_join": false,
  "min_players": 1,
  "max_players": 12,
  "has_display": true,
  "default_settings": {"rounds": 5, "mode": "normal"},
  "settings_schema": {
    "type": "object",
    "properties": {
      "rounds": {"type": "integer", "minimum": 1, "maximum": 20, "title": "Rondes"},
      "mode": {"type": "string", "enum": ["normal", "quick"]}
    },
    "required": ["rounds", "mode"],
    "additionalProperties": false
  }
}
```

Instellingenschema

Een klein deel van JSON Schema. De catalogus controleert schema en standaardwaarden bij het opstarten en weigert alles wat hij niet kent.

- Het schema is een `object` met `properties`, optioneel `required` en verplicht `additionalProperties:false`. Annotaties `title`, `description` en `default` mogen.
- Een property heeft `type` `integer`, `number`, `string` of `boolean`, en optioneel `enum`, `minimum`, `maximum`, `maxLength`, `title`, `description` en `default`.
- Arrays, geneste objecten en `$ref` worden niet ondersteund.

Gebruik absolute paden in de HTML, bijvoorbeeld `/games/my-game/assets/app.js` en `/sdk/mini-engine.js`.

Grenzen en rate limits

Vaste grenzen in 0.2. Overschrijden geeft een fout; niets wordt stilletjes ingekort.

Wat	Grens	Bij overschrijden
HTTP-body	96 KiB	`413 invalid_json`
Websocketbericht	96 KiB	verbinding sluit
Gepubliceerde state (publiek, host en spelers samen)	64 KiB	`413 state_too_large`
Actiedata en instellingen	8 KiB	`400 invalid_request`
Actienaam	1–64 tekens: letters, cijfers, `_ . -`	`400 invalid_request`
`action_id`, `request_id`	hoogstens 64 tekens	`400 invalid_request`
Deduplicatievenster	laatste 256 action-ID's per kamer	oudere ID's tellen weer als nieuw
Spelersnaam, weergavenaam	1–24 tekens	`400 invalid_request`
Standaard `max_players`	maximum van de game, hoogstens 24	hoger mag tot het gamemaximum
Kamers op de server	500	`503 room_limit`
Websocketverbindingen	1024	`503 connection_limit`
Kamer zonder activiteit	12 uur	sluit met `room_expired`
Websocket: auth en stilte	auth binnen 5 s; hoogstens 45 s stil (ping elke 15 s)	`auth_timeout`, `heartbeat_timeout`
Accountsessie	30 dagen	`401 unauthorized`
Gamepackage	2 MiB per bestand, 10 MiB en 4096 bestanden per game	engine start niet

Rate limits

HTTP-tellers lopen per vast venster van zestig seconden. De SDK wacht na `rate_limit` minstens 1,2 seconden voordat hij opnieuw verbindt.

Wat	Limiet	Fout
`POST /api/v1/rooms` en `POST …/join`	60 per minuut per IP-adres	`429 rate_limit`
`POST /api/v1/accounts/register` en `…/login`	10 per minuut per IP-adres	`429 rate_limit`
Websocketberichten	per verbinding 240 per seconde voor de host, 60 voor andere rollen	`error` en sluiten met `rate_limit`

Foutformaat en foutcodes

Elke fout heeft dezelfde vorm, over HTTP en over de websocket. Reageer op de `code`; toon de `message` aan mensen.

```
{"error": {"code": "room_full", "message": "Deze room is vol."}}
```

Over de websocket staat dezelfde fout in `{"type":"error","request_id":"…","error":{…}}`.

Codes

Status	Code	Wanneer
400	`invalid_request`	Een veld klopt niet: naam, instellingen, config, actie of payload.
400	`invalid_json`	Geen geldige JSON, onbekende velden of verkeerd `Content-Type`. Bij een te grote body met status 413.
401	`unauthorized`, `auth_timeout`	Token ontbreekt, is ongeldig, ingetrokken of verlopen; of geen websocket-auth binnen vijf seconden.
403	`forbidden`, `origin_forbidden`	Deze rol mag dit niet, of de browser-origin is niet toegestaan.
404	`room_not_found`	De kamer bestaat niet of is verlopen.
404	`game_not_found`, `game_page_not_found`, `asset_not_found`, `player_not_found`, `not_found`	Game, rolpagina, asset, speler of route bestaat niet.
405	`method_not_allowed`	De route bestaat, de methode niet.
409	`version_conflict`	De versie is intussen veranderd. Lees de nieuwste snapshot.
409	`room_started`, `room_full`, `name_taken`, `already_joined`, `username_taken`	Binnenkomen, klaarzetten of configureren kan niet (meer), de naam of gebruikersnaam is bezet, of dit account heeft al een stoel.
409	`invalid_transition`, `players_not_ready`	Lifecycle in de verkeerde volgorde, of niet iedereen staat klaar.
409	`game_not_running`, `host_offline`	Een spelactie buiten `running`, of bij een hostruntime zonder verbonden host.
409	`session_replaced`	Een nieuwere verbinding gebruikt dit token.
409	`invalid_game_action`, `buzzer_closed`, `teams_locked`, `teams_required`	Spelregels van een servergame.
413	`state_too_large`	Gepubliceerde state groter dan 64 KiB.
429	`rate_limit`	Te veel verzoeken of berichten.
500	`internal_error`	Opslaan mislukte. De oude state blijft staan.
503	`room_limit`, `connection_limit`	De server zit vol. Probeer het later opnieuw.